

Rapport : Sécurisation de Base de Données (questions 27-28)

JUSTINE Tristan

TAYLLAMIN Thomas

1. Question 27 :

1.1 Contexte et Contraintes

L'objectif de cette question était de stocker les salaires d'employés dans une base de données, en garantissant qu'un attaquant ayant un accès complet (en lecture) au serveur de base de données ne puisse jamais lire les montants en clair.

Cependant, il y a une autre contrainte, l'application doit pouvoir :

1. Calculer la somme totale des salaires.
2. Comparer le salaire de deux employés (pour savoir qui gagne le plus).

Le serveur doit réaliser ces opérations lui-même, sur les données chiffrées, sans jamais télécharger toute la base sur le client pour les déchiffrer.

1.2 Choix Algorithmiques

Comme aucune solution simple ne fait les deux opérations simultanément, la solution que l'on a choisi consiste à chiffrer la même information deux fois, avec deux algorithmes différents, pour répondre aux deux besoins :

1. Calculer la somme : Chiffrement Homomorphe (Paillier)
 - **Librairie** : `phe` (Partially Homomorphic Encryption).
 - Le chiffrement de Paillier possède la propriété de l'homomorphisme additif. Si on a $E(a)$ et $E(b)$, le serveur peut multiplier les deux chiffrés : $E(a) \times E(b) = E(a + b)$. Le serveur peut sommer les salaires chiffrés et renvoyer la somme chiffrée au client.
2. Comparer des salaires : Chiffrement Préservant l'Ordre (OPE / ORE)
 - **Librairie** : `pyope`.

- L'espace d'entrée par défaut de pyope était trop petit pour des grandes variables, nous l'avons agrandi avec `ValueRange(0, 2**32-1)`.
- L'OPE chiffre les données sous forme d'entiers très grands, mais si le salaire $A < B$, alors le chiffré $E(A) < E(B)$. Donc le serveur regarde simplement sa base de données avec des opérateurs SQL normaux (`<` ou `>`) sur ces valeurs.

1.3 Architecture et Implémentation (SQLite)

Le système est séparé en deux scripts :

- `client.py` :
C'est le seul composant qui possède les clés secrètes (la clé OPE et les clés de Paillier enregistrées dans `secret.key`). Lorsqu'un utilisateur ajoute un salaire, `client.py` génère le chiffré OPE et le chiffré Paillier, et les envoie au serveur. Lorsqu'il demande une somme, il reçoit un binaire du serveur, et applique sa clé privée pour récupérer l'addition des salaires en clair.
- `server.py` :
Agit comme un Middleware ne connaissant **aucune clé**. Au lancement, il initialise une bdd locale **SQLite** (`q27.db`). S'il doit comparer, il exécute `ORDER BY salaire_ope DESC`. S'il doit sommer, il recupère les `salaire_paillier` de SQLite, les multiplie ensemble (grâce à `phe`), et renvoie la donnée au client. À aucun moment la donnée n'existe en clair sur le serveur.

2. Question 28 : Implémentation d'une Attaque Statistique sur OPE en Base de Données

2.1 Contexte de la faille

Le chiffrement préservant l'ordre (OPE) utilisé pour les comparaisons introduit une vulnérabilité : la fuite de distribution.

Comme il conserve l'ordre de tri, si l'attaquant parvient à récupérer la loi de distribution mathématique du jeu de données, il peut tout retrouver.

2.2 Méthodologie et Preuve de Concept (SQLite)

Pour illustrer le comportement d'un véritable attaquant dans le fichier `stat_attack.py`, le script se divise en deux rôles autour d'une vraie base `entreprise.db`.

1. Infra :

Un jeu de données de salaires chiffrés est généré. L'entreprise chiffre chaque entrée avec l'algorithme OPE et sa clé secrète, puis l'insère dans sa base de données SQLite via `INSERT INTO employees (salaire_ope)`.

2. Exfiltration par l'attaquant :

Comme on considère que l'attaquant a accès à la bdd, il se connecte à SQLite (`sqlite3.connect`) et récupère tout `SELECT salaire_ope FROM employees`. Il obtient une suite d'entiers qui en soit ne représentent rien mais qui conservent l'ordre. On les tri ensuite.

3. L'attaque:

Pour des salaires, l'attaquant va générer une distribution normale et la trier à son tour. Puisque l'ordre OPE = l'ordre du clair, on associe chaque chiffré dans l'ordre à ceux du modèle publique connu.

2.3 Analyse des résultats

Comme le montre la courbe `matplotlib` en résultat d'exécution, la superposition des vrais salaires face aux estimations de l'attaquant possède une erreur très faible dès que l'on a un n assez grand (à partir de 500 c'est très concluant).